Week 4 – Detailed Design Document

Team P3

Cross-Platform Budgeting Application Project

Karunakar Reddy Kaitha, Jordan Lee and John Lim

Computer Science and Information Systems Department, Regis University

MSSE 692: Software Engineering Practicum I

Morgan Worrell

February 8, 2026

Contents

1. Introduction 4
2. Design Goals 4
3. System Overview (From Week 3 Architecture) 4
4. API Specifications 4
4.1 Global API Conventions 4
4.1.1 Base URL 4
4.1.2 Media Types 5
4.1.3 Required Headers 5
4.1.4 Data Types & Constraints 5
4.1.5 Authentication Rules 5
4.2 Standard Error Handling 5
4.2.1 Error Response Schema 5
4.2.2 Error Code Table 6
4.3 Authentication API 8
4.4 Budget Management API 10
4.5 Income API 12
4.6 Expense API 12
4.7 Reports API 13
4.8 Rate Limiting 14
5. Data Model & Schema Design 14
5.1 Entity-Relationship Diagram (ERD) 14
5.2 Database Tables 15
5.3 Indexes 16
5.4 Migration Strategy 16
5.5 Data Access Patterns 17
6. Technology Stack 17
6.1 Infrastructure and Deployment 17
6.2 ADRs for Significant Technology Decisions 17
6.2.1 ADR-005: PostgreSQL Selected as Database 17
6.2.2 ADR-006: FastAPI Selected as Backend Framework 18
6.2.3 ADR-007: React Selected for Frontend Framework 19
6.2.4 ADR-008: SQLAlchemy Selected as ORM 19
6.2.5 ADR-009: JWT-Based Authentication Selected 19
6.2.6 ADR-010: Docker Selected for Containerization 20
7. Component Internal Design 20
7.1 Component Diagram 21
7.2 Class Diagram - Backend Core 22
7.3 Design Patterns Applied 23
7.4 Sequence Diagrams for Critical Workflows 23
7.5 SOLID Principles Compliance 25

8. Quality Attribute Design Strategies 25
8.1 Performance 25
8.2 Security 25
8.3 Testability 26
8.4 Deployability 26
9. Implementation Guidance 26
9.1 Development Setup 26
9.2 Git Workflow 27
9.3 Testing Strategy 27
9.4 Coding Standards 27
10. Conclusion 27

1. Introduction

This document provides implementation-ready design specifications for the Cross-Platform Budgeting Application. It will convert the architectural choices that were established in Week 3, into component-level detailed designs, API contracts, data models, and technology choices, to be used in development in Weeks 5-6. The design meets the functional and quality requirements as stated in Week 2 and complies with the architectural constraints and patterns which were outlined in Week 3.

2. Design Goals

- Define clear, implementation-independent API contracts to enable parallel development.
- Design normalized, scalable data models aligned with reporting and access patterns.
- Select technologies based on system requirements, constraints, and team expertise.
- Use effective design patterns in the components.
- Facilitate the implementation that is secure, testable and maintainable.

3. System Overview (From Week 3 Architecture)

The system is layered and is based on a hybrid modular monolith architecture:

- Presentation Layer (Frontend)
- Application Layer (API Controllers)
- Domain Layer (Business Services)
- Persistence Layer (Repositories & Database)

4. API Specifications

This section defines the formal API contract for the Cross-Platform Budgeting Application. The specifications are implementation-independent and sufficiently detailed to serve as a formal API contract for development and automated integration testing.

4.1 Global API Conventions

4.1.1 Base URL

https://<host>/api/v1

4.1.2 Media Types

Direction	Media Type
Request	application/json
Response	application/json

4.1.3 Required Headers

Header	Required	Description
Content-Type	POST, PUT	Must be application/json
Accept	Yes	Must be application/json
Authorization	Authenticated endpoints	Bearer <JWT>

4.1.4 Data Types & Constraints

Field Type	Definition
UUID	RFC 4122 UUID v4
Monetary Amount	Decimal, 2 fractional digits, must be > 0
Date	ISO 8601 YYYY-MM-DD
Month	ISO 8601 YYYY-MM
Timestamp	ISO 8601 UTC

4.1.5 Authentication Rules

Endpoint Category	Authentication
/auth/register, /auth/login	Public
All other endpoints	JWT Bearer token required

4.2 Standard Error Handling

4.2.1 Error Response Schema

Field	Type	Required	Description
timestamp	string	Yes	ISO-8601 UTC
status	integer	Yes	HTTP status code
error	string	Yes	HTTP reason phrase

errorCode	string	Yes	See Error Code Table
message	string	Yes	Human-readable summary
path	string	Yes	Request path
details	array	No	Field-level validation errors

Example (JSON)

```
{
  "timestamp": "2026-02-01T10:15:30Z",
  "status": 400,
  "error": "Bad Request",
  "errorCode": "VAL-001",
  "message": "Validation failed",
  "path": "/api/v1/expenses",
  "details": [
    { "field": "amount", "issue": "must be greater than 0" }
  ]
}
```

4.2.2 Error Code Table

Error Code	HTTP Status	Error	Default Message	When It Occurs	Client Action	Retryable
AUTH-001	401	Unauthorized	Missing or invalid token	No Authorization: Bearer … header, malformed JWT, or token verification fails	Re-authenticate / refresh token	No

AUTH-002	401	Unauthorized	Token expired	JWT is valid but expired	Refresh token / re-login	Yes
AUTH-003	403	Forbidden	Insufficient permissions	User authenticated but not authorized for resource/action	Show access denied	No
AUTH-004	401	Unauthorized	Invalid credentials	Login email/password incorrect	Prompt user to retry login	No
USR-001	409	Conflict	Email already registered	Register called with existing email	Ask user to login instead	No
USR-002	404	Not Found	User not found	UserId does not exist (admin/debug)	Display error	No
VAL-001	400	Bad Request	Validation failed	Missing/invalid request fields, wrong types/formats	Correct request payload	No
VAL-002	400	Bad Request	Invalid date format	Date not in expected format (e.g., YYYY-MM-DD)	Correct request payload	No
BUD-001	404	Not Found	Budget not found	budgetId not found	Inform user / refresh view	No
BUD-002	409	Conflict	Budget already	Attempt to create	Prompt to edit	No

			exists for month	duplicate month budget for same user	existing budget	
INC-001	404	Not Found	Income not found	Income record not found (if endpoints added later)	Inform user	No
EXP-001	404	Not Found	Expense not found	Expense record not found (if endpoints added later)	Inform user	No
EXP-002	400	Bad Request	Invalid amount	Amount is zero/negative or exceeds allowed max	Correct request	No
RPT-001	400	Bad Request	Unsupported report month	Month missing/invalid or out of allowed range	Correct query param	No
SYS-001	500	Internal Server Error	Unexpected server error	Unhandled exception	Show generic error	Yes
SYS-002	503	Service Unavailable	Database unavailable	DB timeout/outage	Retry with backoff	Yes

4.3 Authentication API

POST /auth/register

Authentication

- None (Public)

Request Body

Field	Type	Required	Constraints
email	string	Yes	Valid email, max 254 chars
password	string	Yes	≥8 chars, upper/lower/number/symbol
name	string	Yes	1–100 chars

Success Response — 201 Created

Field	Type
userId	UUID
message	string

Error Responses

Status	Error Code	Condition
400	VAL-001	Invalid field(s)
409	USR-001	Email already registered

POST /auth/login

Authentication

- None (Public)

Request Body

Field	Type	Required
email	string	Yes
password	string	Yes

Success Response — 200 OK

Field	Type
accessToken	string
tokenType	string (Bearer)
expiresIn	integer (seconds)

Error Responses

Status	Error Code	Condition
401	AUTH-004	Invalid credentials

4.4 Budget Management API

POST /budgets

Authentication

- JWT Required

Request Body

Field	Type	Required	Constraints
month	string	Yes	YYYY-MM
totalAmount	number	Yes	> 0

Success Response — 201 Created

Field	Type
budgetId	UUID
month	string
totalAmount	number
createdAt	timestamp

Error Responses

Status	Error Code	Condition
400	VAL-001	Invalid month or amount
409	BUD-002	Budget already exists for month

GET /budgets/{budgetId}

Path Parameters

Name	Type	Required

budgetId	UUID	Yes

Success Response — 200 OK

Field	Type
budgetId	UUID
month	string
totalAmount	number
createdAt	timestamp
updatedAt	timestamp

Error Responses

Status	Error Code
404	BUD-001
401	AUTH-001

PUT /budgets/{budgetId}

Authentication

- JWT Required

Request Body

Field	Type	Required
totalAmount	number	Yes

Success Response — 200 OK

Field	Type
budgetId	UUID
month	string
totalAmount	number
updatedAt	timestamp

4.5 Income API

POST /incomes

Authentication

- JWT Required

Request Body

Field	Type	Required
amount	number	Yes
source	string	Yes
date	string	Yes (YYYY-MM-DD)

Success Response — 201 Created

Field	Type
incomeId	UUID
amount	number
source	string
date	string
createdAt	timestamp

4.6 Expense API

POST /expenses

Authentication

- JWT Required

Request Body

Field	Type	Required
amount	number	Yes
category	string	Yes
date	string	Yes
note	string	No

Success Response — 201 Created

Field	Type
expenseId	UUID
amount	number
category	string
date	string
note	string
createdAt	timestamp

Error Responses

Status	Error Code
400	EXP-002
400	VAL-001

4.7 Reports API

GET /reports/summary

Authentication

- JWT Required

Query Parameters

Name	Type	Required
month	string	Yes (YYYY-MM)

Success Response — 200 OK

Field	Type
month	string
totalIncome	number
totalExpenses	number
net	number
byCategory	array

generatedAt	timestamp

Error Responses

Status	Error Code
400	RPT-001

4.8 Rate Limiting

Rate limiting is not required for the MVP and is therefore not enforced in the initial implementation. If introduced, it will be applied at the API ingress layer (reverse proxy / API gateway) on a per-user (authenticated) and per-IP (unauthenticated) basis, returning 429 Too Many Requests using the standard error response schema with a dedicated error code and a Retry-After header.

5. Data Model & Schema Design

5.1 Entity-Relationship Diagram (ERD)

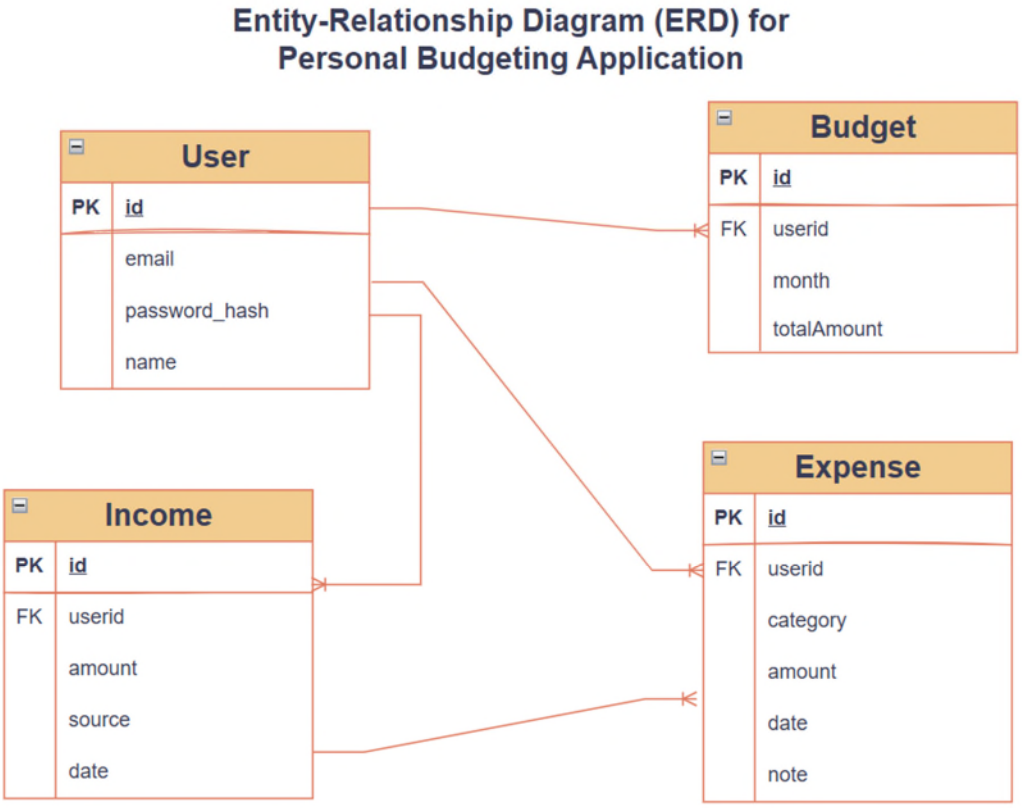

Figure 1: Cross-Platform Budgeting Application Entity-Relationship Diagram (ERD).

This ERD depicts the main entities (User, Budget, Income, Expense) and their connection. Each User may own multiple Budgets, Incomes, and Expenses, while each Budget, Income, and Expense is associated with exactly one User. The diagram ensures referential integrity and supports efficient querying for monthly summaries and reports.

5.2 Database Tables

User

Column	Type	Constraints
id	UUID	PK
email	VARCHAR	UNIQUE, NOT NULL
password_hash	VARCHAR	NOT NULL
name	VARCHAR	NOT NULL

Budget

Column	Type	Constraints
id	UUID	PK
userid	UUID	FK Users(id)
month	VARCHAR	
totalamount	DECIMAL	

Income

Column	Type	Constraints
id	UUID	PK
userid	UUID	FK
amount	DECIMAL	
source	VARCHAR	
date	DATE	

Expense

Column	Type	Constraints
id	UUID	PK
userid	UUID	FK
category	VARCHAR	
amount	DECIMAL	
date	DATE	
note	VARCHAR	

5.3 Indexes

The following indexes are defined to support common query patterns and ensure efficient performance for user-scoped and time-based queries:

- **users(email)**
  Supports fast lookup during authentication and user registration workflows, ensuring uniqueness and efficient login validation.
- **expenses(userid, date)**
  Optimizes retrieval of expense records by user and date range, which is frequently required for monthly summaries and reporting.
- **incomes(userid, date)**
  Enables efficient aggregation and filtering of income records by user and reporting period.

These indexes align with the application's primary access patterns and reporting requirements while minimizing query latency.

5.4 Migration Strategy

- Versioned SQL migrations Flyway provides automated SQL migrations.
- Schema migrations are executed automatically at application startup to ensure schema consistency across environments.

Schema evolution will follow a controlled, versioned approach using incremental migration scripts. Each migration is backward-compatible and traceable, allowing safe

application upgrades, rollback capability, and consistent schema management across development, testing, and production environments.

5.5 Data Access Patterns

The data model is designed to support common access patterns including user-scoped CRUD operations, date-range queries for incomes and expenses, and aggregated monthly budget and reporting queries, with indexes and foreign key relationships ensuring efficient execution.

6. Technology Stack

Layer	Technology
Backend	Python FastAPI
Frontend	React / React Native
Database	PostgreSQL
ORM	SQLAlchemy
Authentication	OAuth2 + JWT
Build Tools	Poetry / pip
Testing	PyTest
API Docs	OpenAPI (Swagger)
Containerization	Docker

6.1 Infrastructure and Deployment

The application is designed to be deployed using containerized services on a cloud-compatible hosting platform, with environment-based configuration, centralized logging, health check endpoints, and automated CI/CD pipelines to support continuous integration, testing, and controlled promotion of builds across environments.

6.2 ADRs for Significant Technology Decisions

6.2.1 ADR-005: PostgreSQL Selected as Database

Context:

The application requires a reliable and scalable data storage solution that supports relational data modeling, transactional integrity, and efficient querying for budgeting, reporting, and historical financial data.

Decision:

PostgreSQL is selected as the primary relational database management system.

Rationale:

PostgreSQL provides strong support for relational integrity, advanced indexing, transactional consistency (ACID compliance), and rich querying capabilities, making it well suited for normalized financial data and reporting workloads.

Consequences:

The data model will leverage relational constraints and indexes to enforce consistency and optimize queries, and database migrations will be managed to support controlled schema evolution.

6.2.2 ADR-006: FastAPI Selected as Backend Framework

Context:

The backend requires a high-performance web framework capable of supporting asynchronous request handling, clear API contracts, and rapid development with strong tooling support.

Decision:

FastAPI is selected as the backend framework for implementing RESTful APIs.

Rationale:

FastAPI offers native asynchronous support, automatic OpenAPI documentation generation, strong request validation, and seamless integration with Python’s ecosystem, aligning with the project’s performance, maintainability, and developer productivity goals.

Consequences:

The backend will follow an async-first design, use schema-driven API validation, and expose standardized API documentation, enabling parallel development and easier integration testing.

6.2.3 ADR-007: React Selected for Frontend Framework

Context:

The application requires a modern, component-based frontend capable of supporting responsive user interfaces and integration with RESTful APIs.

Decision:

React is selected as the frontend framework for the web client.

Rationale:

React provides a mature ecosystem, strong community support, reusable component architecture, and efficient state management patterns that align with the project's scalability and maintainability requirements.

Consequences:

The frontend will follow a component-driven design and leverage standard React tooling, enabling future extension and cross-platform alignment with React Native if required.

6.2.4 ADR-008: SQLAlchemy Selected as ORM

Context:

The backend requires an abstraction layer for database access that supports relational modeling, migrations, and testability.

Decision:

SQLAlchemy is selected as the Object-Relational Mapping (ORM) framework.

Rationale:

SQLAlchemy provides explicit control over database interactions, supports complex queries, integrates well with PostgreSQL, and aligns with the repository pattern used in the application design.

Consequences:

Database access logic is centralized and abstracted, improving maintainability and enabling easier mocking and testing of persistence logic.

6.2.5 ADR-009: JWT-Based Authentication Selected

Context:

The system requires stateless authentication suitable for a RESTful API and scalable client-server interactions.

Decision:

JWT-based authentication using OAuth2 flows is selected for user authentication and authorization.

Rationale:

JWT enables stateless authentication, reduces server-side session management, integrates naturally with FastAPI, and supports secure role-based access control.

Consequences:

Clients must securely manage tokens, and token expiration and refresh strategies must be enforced to maintain security.

6.2.6 ADR-010: Docker Selected for Containerization

Context:

The application must be deployable consistently across development, testing, and production environments.

Decision:

Docker is selected as the containerization platform for all application components.

Rationale:

Docker ensures environment consistency, simplifies dependency management, and supports scalable deployment workflows across different hosting platforms.

Consequences:

The application will be packaged and deployed as container images, requiring container-aware development and operational practices.

7. Component Internal Design

7.1 Component Diagram

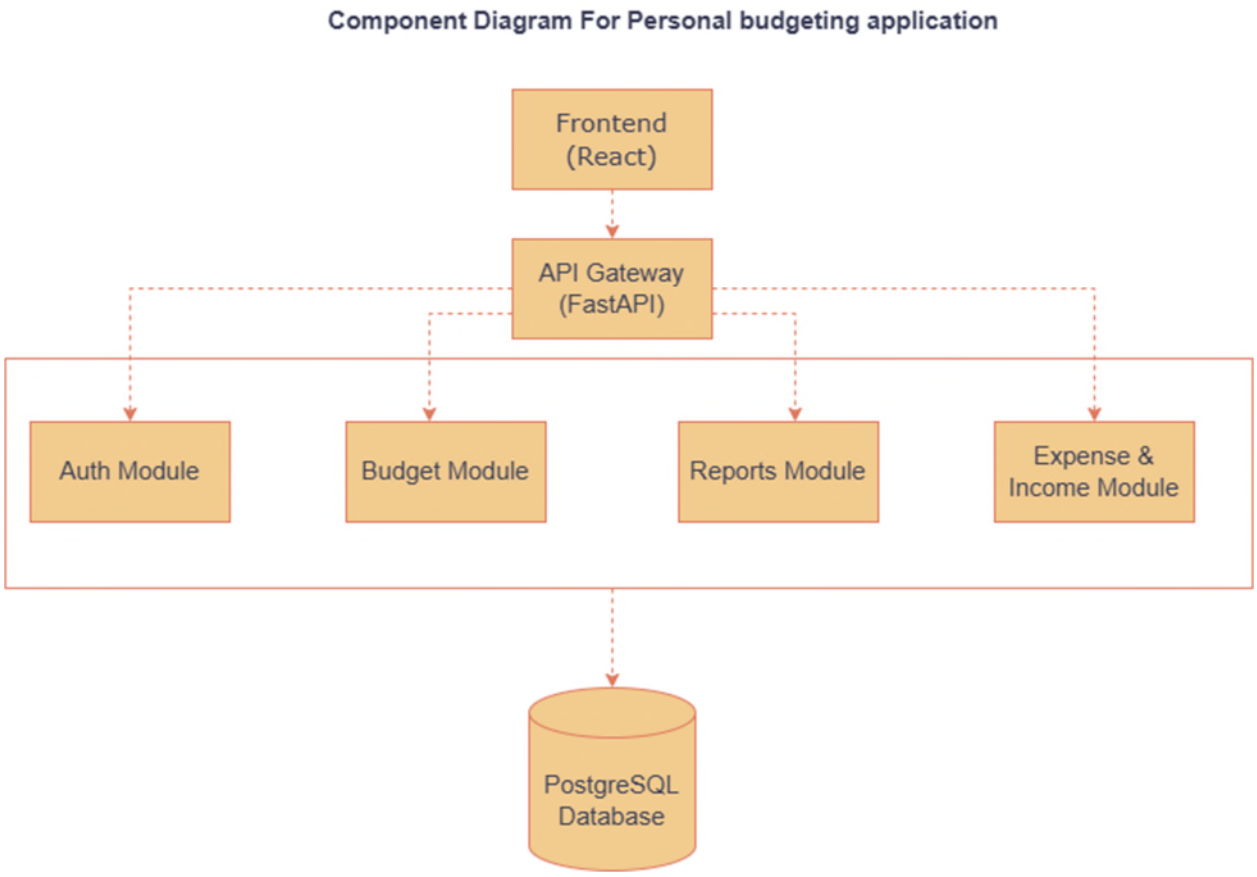

Figure 2: Backend Component Architecture

The diagram above in Figure 2 demonstrates the interactions of the major subsystems and the business logic and persistence separation.

7.2 Class Diagram - Backend Core

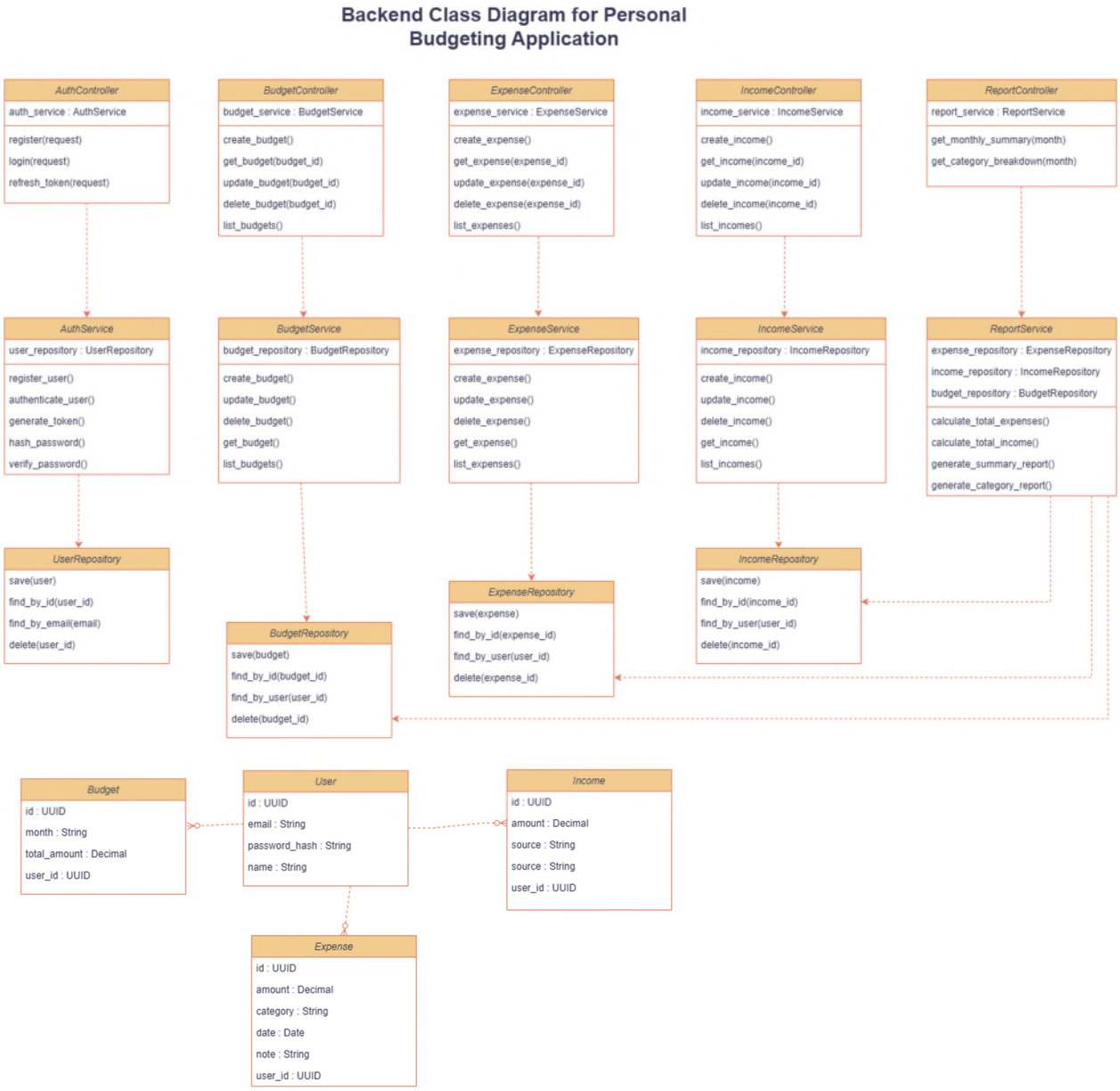

Figure 3: Backend Class Diagram for Cross-Platform Budgeting Application

The structure of the class hierarchy of the backend is shown in Figure 3. Services contain business logic, Repositories are abstract database operations, and Domain Models are persistent entities and are handled by controllers. This design establishes a separation of concerns and maintains maintainability, testability as well as scalability.

7.3 Design Patterns Applied

- Repository Pattern – abstracts persistence concerns.
- Strategy Pattern – encapsulates report calculation algorithms.
- Factory Pattern – centralizes creation of report generators.
- Dependency Injection – decouples services from concrete implementations.

7.4 Sequence Diagrams for Critical Workflows

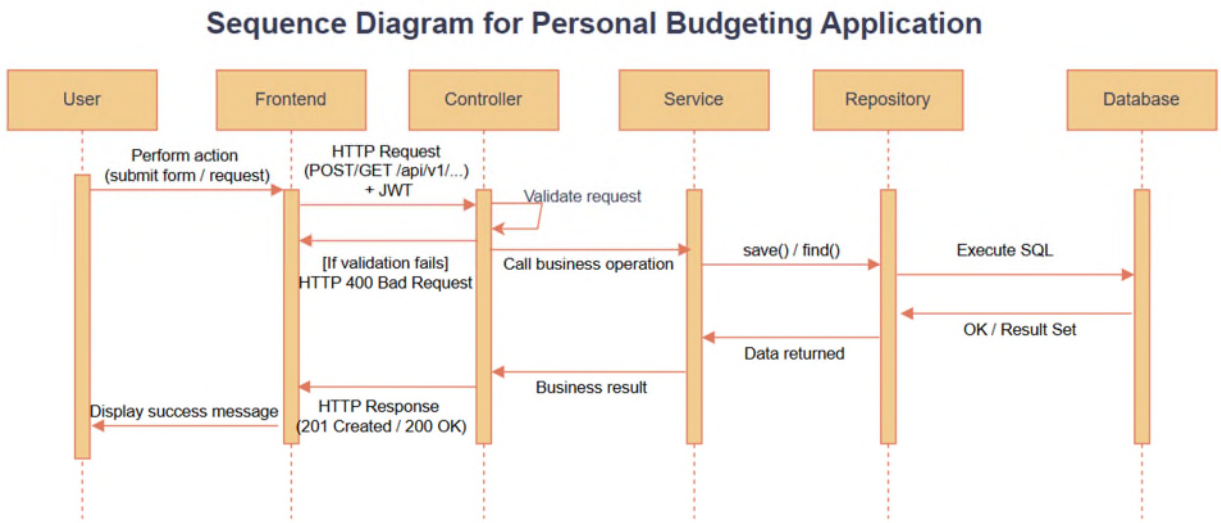

Figure 4: End-to-End API Request Processing Sequence

The sequence diagram in Figure 4 above illustrates the standard end-to-end processing of an API request within the system. It shows the interaction between the frontend client, API controller, authentication middleware, domain service, repository layer, and database. The diagram emphasizes the layered architecture, separation of concerns, the request–response lifecycle that is common across authenticated API operations, and serves as a reference interaction pattern for most authenticated RESTful endpoints.

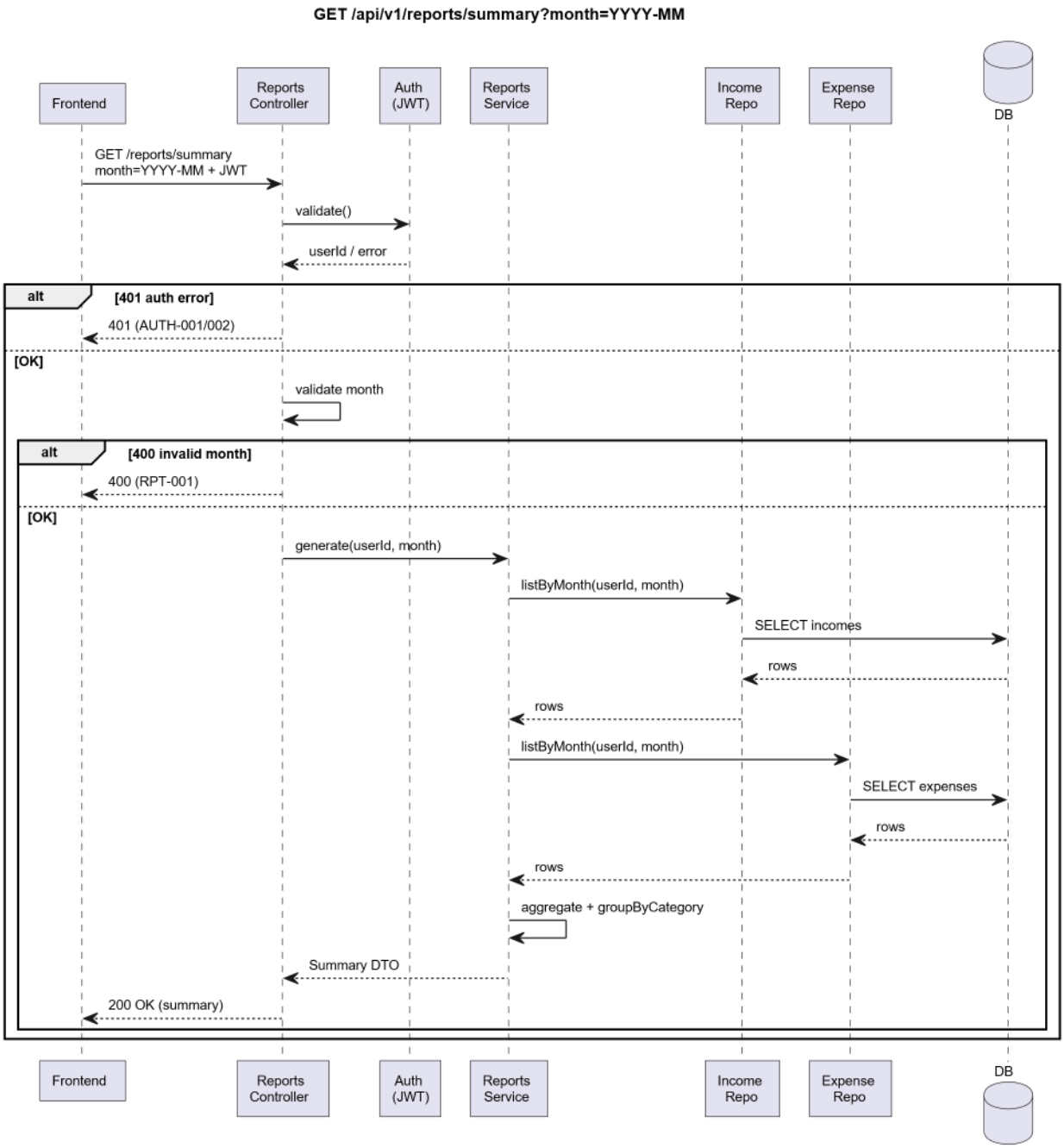

Figure 5: Monthly Summary Report Generation (GET /reports/summary)

The sequence diagram in Figure 5 above illustrates the end-to-end workflow for generating a monthly summary report. The frontend client invokes GET /reports/summary?month=YYYY-MM using a valid JWT. The controller validates inputs and authentication context, then delegates to the ReportsService. The service queries

IncomeRepository and ExpenseRepository for the specified user and date range, performs aggregation (total income, total expenses, net) and category grouping, and returns a normalized response DTO to the controller. The controller returns a 200 OK response. Invalid month inputs result in a 400 (RPT-001), and missing/invalid tokens result in 401 (AUTH-001/AUTH-002) using the standard error schema.

7.5 SOLID Principles Compliance

The component design follows SOLID principles to promote maintainability, extensibility, and testability. Single Responsibility is enforced by separating concerns across controllers, services, repositories, and domain models, each with a clearly defined role. The Open/Closed Principle is supported through the use of Strategy and Factory patterns for report generation, allowing new report types to be added without modifying existing logic. Dependency Inversion is achieved via dependency injection, enabling services to depend on abstractions rather than concrete implementations, which also improves testability and flexibility.

8. Quality Attribute Design Strategies

The following design strategies demonstrate how the system architecture and component design address the quality attribute scenarios defined in Week 2.

8.1 Performance

- Async endpoints
- Indexed queries
- Pagination

Performance requirements are addressed through asynchronous API endpoints, indexed database queries aligned with common access patterns, pagination for large result sets, and optional caching at the application or HTTP layer to reduce repeated computation and database load.

8.2 Security

- JWT tokens

- Password hashing (bcrypt)
- Role-based authorization

Security is enforced through JWT-based authentication and authorization, secure password hashing, strict request validation, standardized error handling, and controlled access to resources to prevent unauthorized or malformed requests.

8.3 Testability

- Dependency injection
- Mock repositories
- PyTest fixtures

Testability is achieved by enforcing clear separation of concerns, using dependency injection and repository abstractions, and supporting mocking and isolated testing of services through automated unit and integration tests.

8.4 Deployability

- Docker containers
- Environment variables
- Logging with Loguru

Application deployability is supported through containerized deployment, environment-based configuration management, structured logging, and the inclusion of health check endpoints and monitoring hooks to enable runtime observability and operational readiness.

9. Implementation Guidance

9.1 Development Setup

- Python 3.11
- Node 18
- Docker Desktop

The development environment is configured using standardized runtime versions, containerized services, and local tooling to enable consistent setup across team members and ensure parity between development and deployment environments.

9.2 Git Workflow

- main
- develop
- feature/* branches

All changes are developed on feature branches and merged via pull requests that require peer review and successful automated tests before integration into the develop branch, ensuring code quality and traceability of changes.

9.3 Testing Strategy

- Unit tests per service
- API integration tests
- Open API Contract testing.

The testing strategy includes unit tests for core services, integration tests for API endpoints, and contract-based testing using the OpenAPI specification to validate correctness and prevent interface regressions.

9.4 Coding Standards

Coding standards follow consistent naming conventions, standardized formatting, and clear package organization aligned with the layered architecture, with automated linters and formatters used to enforce readability, consistency, and maintainability across the codebase.

10. Conclusion

The present design document gives full specifications that can be efficiently implemented during Weeks 5-6. The design follows architectural constraint, and meets the functional and quality requirement, and also in compliance with maintainable growth.